

Report: LLM-Based Entity Extraction for Shared Housing

1. Introduction

This assignment focuses on using Large Language Models for entity extraction in a shared-housing domain. The main goal is to extract structured information from free-text apartment descriptions. In this notebook, the extraction task is limited to two entity categories: **Apartments** and **Amenities**.

Apartment listings usually contain useful information written in natural language. For example, a description may mention that an apartment includes Wi-Fi, parking, heating, a balcony, or other facilities. Instead of manually identifying these details, the system uses an LLM to extract them automatically and return them in a structured JSON format.

The extracted entities can later be used to populate a knowledge graph. For example, an apartment entity can become a node in Neo4j, and each amenity can also become a node connected to the apartment through a relationship such as `HAS_AMENITY`. Therefore, this task can be seen as the first step toward building a knowledge graph and supporting a GraphRAG question-answering system.

The general pipeline is:

Apartment Description → **LLM Entity Extraction** → **Structured JSON Output** → **Evaluation against Ground Truth**

2. Dataset and Task Definition

The notebook uses a small manually annotated dataset with **12 housing descriptions**. Each example contains the original text and the expected entities annotated by humans.

The two target entity types are:

Apartment:

This category includes mentions of apartments or housing units. Examples include “apartment”, “studio”, “shared apartment”, “two-bedroom apartment”, and similar expressions.

Amenity:

This category includes facilities, services, utilities, or features available in the apartment. Examples include “Wi-Fi”, “parking”, “heating”, “balcony”, “garden”, “laundry”, “elevator”, “storage”, “air conditioning”, “game room”, and “ironing equipment”.

An important detail is that amenities are treated as an **open-vocabulary category**. This means that the model should not only extract amenities from a fixed predefined list. If a new amenity appears in the text, the model should extract it as long as it is clearly available to residents.

This is useful because real apartment descriptions can contain many different types of amenities, and it is not realistic to define all possible amenities in advance.

3. System Setup

The notebook begins by importing the required Python libraries and loading environment variables. The OpenAI API key and model name are loaded from a `.env` file. This is a better practice than writing the API key directly inside the notebook, because it keeps sensitive information separate from the code.

The main environment variables used are:

- `OPENAI_API_KEY`
- `OPENAI_MODEL`

The notebook also handles the dataset path flexibly. It searches for the dataset file in more than one possible location, which makes the notebook easier to run from different folders or machines.

This setup makes the code cleaner, more reusable, and safer.

4. LLM Extraction Prompt

A central part of the system is the prompt used for entity extraction. The prompt gives the LLM clear instructions about what to extract and how to format the output.

The model is instructed to extract only two types of entities:

- Apartments
- Amenities

The output must be returned as structured JSON. This is important because JSON output can be easily processed by Python and evaluated automatically.

The prompt also includes important rules. First, the model should extract entities exactly as they appear in the text. Second, it should not hallucinate entities that are not explicitly mentioned. Third, it should not infer information that is only implied. Fourth, negated amenities should not be extracted as positive amenities.

For example, if a description says:

“The apartment has Wi-Fi and parking.”

then the model should extract “Wi-Fi” and “parking” as amenities.

However, if a description says:

“The apartment does not have parking.”

then “parking” should not be extracted as an available amenity.

This rule is important because the task is not simply to detect amenity words. The system must understand whether the amenity is actually available.

5. JSON Parsing and Robustness

The notebook includes a helper function to parse the JSON response returned by the LLM. This is necessary because LLMs sometimes return extra text before or after the JSON object, even when they are instructed to return only JSON.

The parser first tries to load the full response directly as JSON. If this fails, it searches inside the response text for a valid JSON object and then parses that object.

This makes the extraction pipeline more robust. Without this step, the notebook could fail even if the model extracted the correct entities but added a small explanation around the JSON response.

The JSON parser therefore helps the system handle minor formatting inconsistencies in LLM outputs.

6. Negation Policy Check

Before evaluating the LLM, the notebook checks whether the manually annotated ground truth follows the same negation policy as the prompt.

Since the prompt tells the model not to extract negated amenities, the ground truth should also exclude amenities that are mentioned as unavailable.

The notebook checks patterns such as:

- “no amenity”
- “without amenity”
- “does not include amenity”
- “does not offer amenity”
- “does not have amenity”

The result of this check is that the ground-truth negation policy is consistent. This means that the dataset annotations are aligned with the extraction rules.

This is an important validation step. If the ground truth included negated amenities while the model was instructed to exclude them, then the evaluation would be unfair.

7. Open-Vocabulary Extraction Test

The notebook also tests whether the model can extract amenities outside a small fixed list. This is done with the sentence:

“The apartments include a game room and ironing equipment.”

The expected amenities are:

- game room
- ironing equipment

This test is useful because these amenities are less common than simple examples such as Wi-Fi or parking. The model successfully extracts both “game room” and “ironing equipment”.

This result shows that the prompt supports open-vocabulary extraction. The model is not limited to a predefined list of amenities, but can recognize new amenity types when they appear clearly in the text.

8. Evaluation Methodology

The system is evaluated using standard information extraction metrics:

- Precision
- Recall
- F1-score

Evaluation is performed separately for apartment entities and amenity entities.

Before comparing predictions with ground truth, both are normalized. The normalization process includes lowercasing and removing extra spaces. This avoids penalizing the model for small formatting differences.

The evaluation is based on three values:

True Positives: entities correctly extracted by the model.

False Positives: entities extracted by the model but not present in the ground truth.

False Negatives: entities present in the ground truth but missed by the model.

Precision measures how many extracted entities were correct. High precision means the model does not produce many incorrect extractions.

Recall measures how many ground-truth entities were found. High recall means the model does not miss many correct entities.

F1-score combines precision and recall into one balanced metric.

This evaluation method is appropriate because the task is a standard entity extraction problem.

9. Results

The final evaluation was run on the complete annotated dataset.

For **Apartment** entities, the results were:

- Precision: **0.833**
- Recall: **0.792**
- F1-score: **0.806**

For **Amenity** entities, the results were:

- Precision: **0.847**
- Recall: **0.847**
- F1-score: **0.847**

The results show that the system performs reasonably well on both entity categories.

The amenity extraction results are slightly better and more balanced. Precision and recall are equal, which means the model has a good balance between avoiding incorrect amenities and finding the correct ones.

For apartment entities, precision is slightly higher than recall. This means that when the model extracts an apartment entity, it is usually correct, but it sometimes misses apartment mentions that exist in the ground truth.

Overall, the results show that the LLM is able to extract useful structured information from housing descriptions with solid accuracy.

10. Optional LLM-as-a-Judge

The notebook also includes an optional LLM-as-a-judge component. This component can compare the model's extracted entities with the human annotations and judge whether each extraction is correct.

However, this is not used as the main evaluation method. The main evaluation is based on deterministic comparison with the manually annotated ground truth.

This is a good design choice because the ground truth should remain the main reference. The LLM judge can be useful for additional qualitative analysis, but it should not replace standard metrics such as precision, recall, and F1-score.

11. Strengths of the System

One strength of the system is that it uses a clear and structured prompt. The prompt defines the entity types, explains the extraction rules, and gives specific instructions about negation and open-vocabulary amenities.

Another strength is that the notebook includes validation checks before the final evaluation. The negation policy check ensures that the ground truth is consistent with the prompt. The open-vocabulary test confirms that the model can extract new amenities beyond a fixed list.

The use of JSON output is also important. It makes the extracted entities easy to process, evaluate, and later transform into graph nodes and relationships.

Finally, the system is directly connected to the broader knowledge graph task. The extracted apartment and amenity entities can be used to automatically build part of a Neo4j knowledge graph.

12. Limitations

The main limitation of the system is the small dataset size. The dataset contains only 12 examples, so the results are useful for this assignment but may not fully represent real-world performance.

Another limitation is that the evaluation uses exact matching after normalization. This can penalize extractions that are semantically correct but written differently from the ground truth. For example, “internet access” and “Wi-Fi” may sometimes refer to similar concepts, but exact matching treats them as different strings.

Also, the system extracts only two entity types. A more complete housing extraction system could also extract rent prices, locations, house rules, owners, roommates, bedrooms, bathrooms, and pet policies.

Finally, the output depends on the LLM. Although the JSON parser improves robustness, model responses can still vary depending on the model version and prompt.

13. Future Improvements

Several improvements could make the system stronger.

First, the dataset could be expanded with more examples and more varied apartment descriptions. This would allow a more reliable evaluation.

Second, more entity types could be added, such as rent, location, owner, roommate, number of bedrooms, number of bathrooms, and rules.

Third, the evaluation could support synonym-aware matching. This would make the evaluation fairer when the model extracts a semantically correct entity using slightly different wording.

Fourth, relation extraction could be added. Instead of only extracting entities, the system could also extract relationships such as:

- Apartment has Amenity
- Apartment located in Location
- Owner owns Apartment
- Apartment has Rule

This would make it possible to automatically populate a knowledge graph from raw apartment descriptions.

14. Conclusion

This assignment demonstrates how an LLM can be used for entity extraction in a shared-housing domain.

The system extracts apartment and amenity entities from unstructured housing descriptions and evaluates them against a manually annotated dataset.

The final results are solid, with an apartment F1-score of **0.806** and an amenity F1-score of **0.847**. The open-vocabulary test also shows that the model can extract new amenities such as “game room” and “ironing equipment”.

Overall, the notebook provides a clear baseline for LLM-based information extraction. The extracted entities can be used as the first step toward automatic knowledge graph construction and future GraphRAG applications.